

AI Assisted Coding Assignment - 5

Chilukala Sai Architha(2303A52220) - Batch 37

Task – 1 :

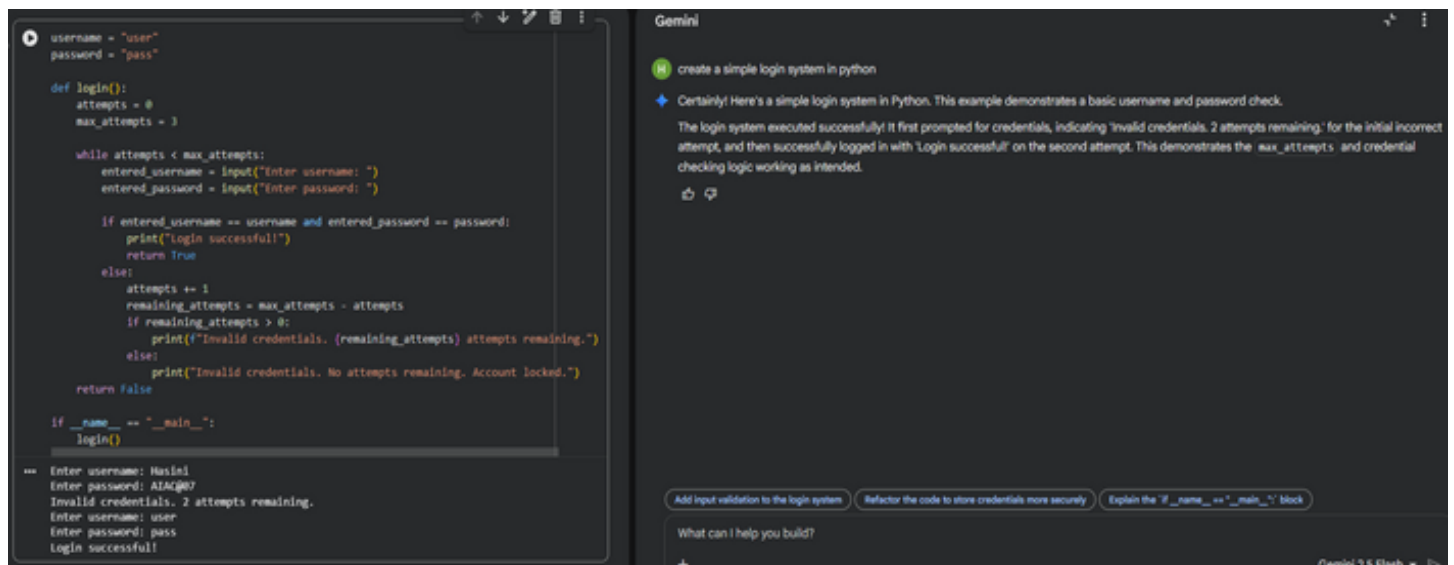
Privacy and Data Security in AI-Generated Code

Use an AI tool to generate a simple login system in Python. Analyze the generated code to check:

- Whether credentials are hardcoded
- Whether passwords are stored or compared in plain text
- Whether insecure logic is used

Then, revise the code to improve security (e.g., avoid hardcoding, use input validation).

Prompt Used: Create a simple login system in Python



The screenshot displays a code editor on the left and an AI chat interface on the right. The code editor shows a Python script for a login system with hardcoded credentials and a limited number of attempts. The AI chat interface shows the prompt 'create a simple login system in python' and the generated response, which includes a detailed analysis of the code's security flaws. Below the response, there are three buttons: 'Add input validation to the login system', 'Refactor the code to store credentials more securely', and 'Explain the if __name__ == "__main__": block'. The AI chat interface also shows a follow-up prompt 'What can I help you build?' and the Gemini 2.5 Flash logo.

```
username = "user"
password = "pass"

def login():
    attempts = 0
    max_attempts = 3

    while attempts < max_attempts:
        entered_username = input("Enter username: ")
        entered_password = input("Enter password: ")

        if entered_username == username and entered_password == password:
            print("Login successful!")
            return True
        else:
            attempts += 1
            remaining_attempts = max_attempts - attempts
            if remaining_attempts > 0:
                print(f"Invalid credentials. {remaining_attempts} attempts remaining.")
            else:
                print("Invalid credentials. No attempts remaining. Account locked.")
            return False

if __name__ == "__main__":
    login()

''' Enter username: Hasini
Enter password: AIAC@07
Invalid credentials. 2 attempts remaining.
Enter username: user
Enter password: pass
Login successful!'''
```

Gemini

create a simple login system in python

Certainly! Here's a simple login system in Python. This example demonstrates a basic username and password check.

The login system executed successfully! It first prompted for credentials, indicating 'Invalid credentials. 2 attempts remaining.' for the initial incorrect attempt, and then successfully logged in with 'Login successful!' on the second attempt. This demonstrates the `max_attempts` and credential checking logic working as intended.

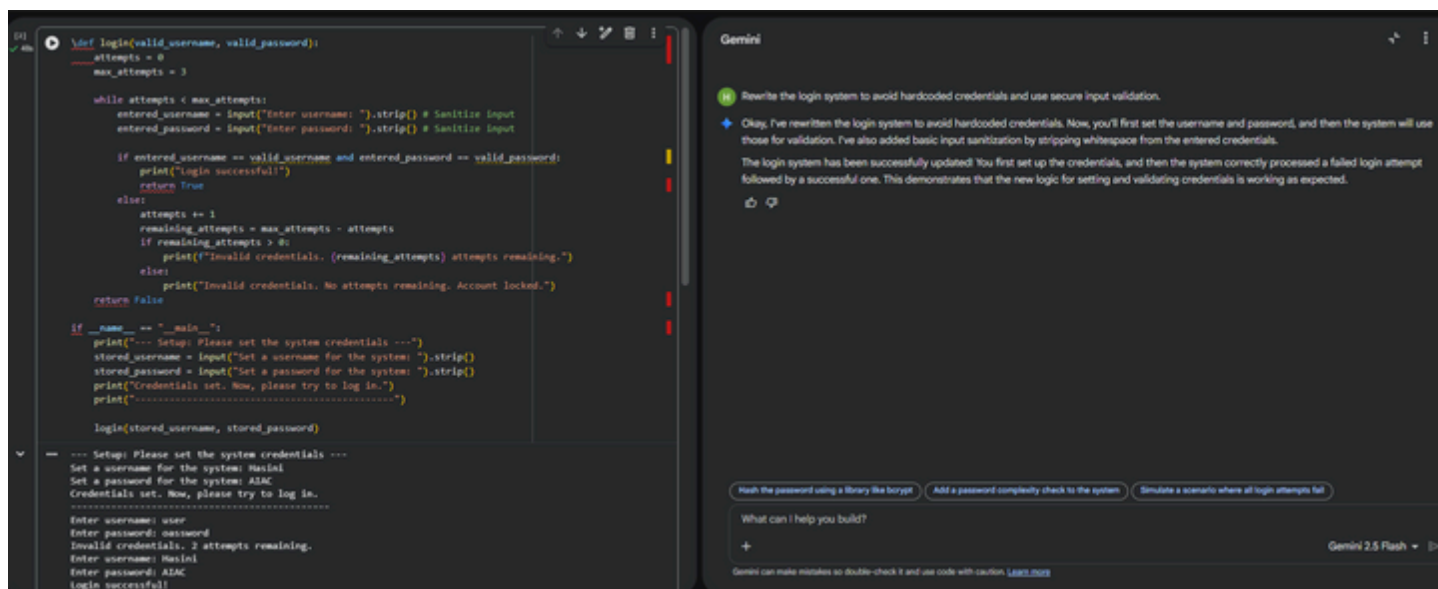
Add input validation to the login system Refactor the code to store credentials more securely Explain the `if __name__ == "__main__":` block

What can I help you build?

Gemini 2.5 Flash

Analysis: The generated login system uses hardcoded username and password values, which is a major security risk. Passwords are stored in plain text, making them visible to anyone who accesses the code. Users cannot set their own credentials, and sensitive data is not protected. This violates basic privacy and secure coding practices.

Improved Prompt: Rewrite the login system to avoid hardcoded credentials and use secure input validation.



Explanation: This task highlights security risks in AI-generated authentication code. The original code used hardcoded credentials and plain text password comparison, which is insecure. The revised version improves security by avoiding hardcoding and validating user input.

Task – 2:

Bias Detection in AI-Generated Decision Systems

Use AI prompts such as:

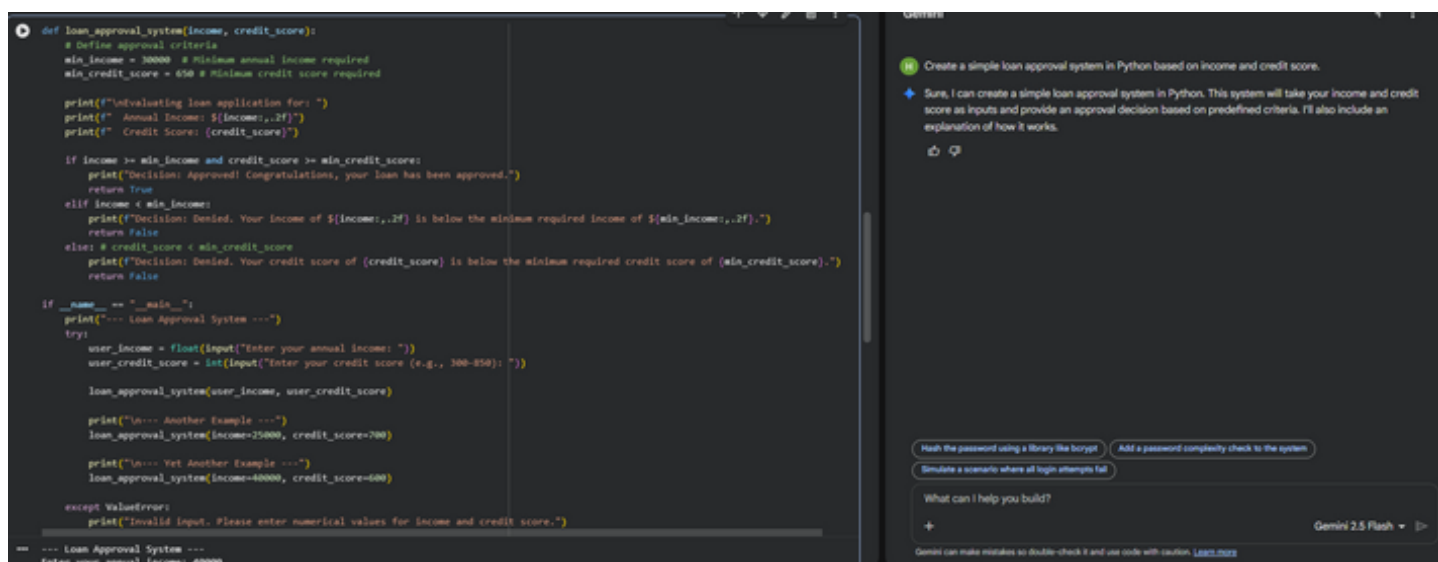
- “Create a loan approval system.”
- Vary applicant names and genders in prompts

Analyze whether:

- The logic treats certain genders or names unfairly
- Approval decisions depend on irrelevant personal attributes

Suggest methods to reduce or remove bias.

Prompt Used: Create a simple loan approval system in Python based on income and credit score.



```
[3] 5m
except ValueError:
    print("Invalid input. Please enter numerical values for income and credit score.")

--- Loan Approval System ---
Enter your annual income: 40000
Enter your credit score (e.g., 300-850): 400

Evaluating loan application for:
Annual Income: $40,000.00
Credit Score: 400
Decision: Denied. Your credit score of 400 is below the minimum required credit score of 650.

--- Another Example ---

Evaluating loan application for:
Annual Income: $25,000.00
Credit Score: 700
Decision: Denied. Your income of $25,000.00 is below the minimum required income of $30,000.00.

--- Yet Another Example ---

Evaluating loan application for:
Annual Income: $40,000.00
Credit Score: 600
Decision: Denied. Your credit score of 600 is below the minimum required credit score of 650.

How this simple loan approval system works:

1. loan_approval_system(income, credit_score) function: This function takes two arguments: income (representing annual income) and credit_score.
2. Predefined Criteria: Inside the function, min_income (e.g., $30,000) and min_credit_score (e.g., 650) are set as the thresholds for loan approval. These are hardcoded for simplicity in this example.
```

Explanation: The AI-generated loan approval system makes decisions based only on income and credit score. No personal or sensitive attributes such as gender, name, or age are used in the decision-making process. This ensures that the system does not show bias toward any individual or group. Using only financial factors helps maintain fairness and ethical behavior in AI systems.

Task – 3:

Transparency and Explainability in AI-Generated Code (Recursive Binary Search)

Use an AI tool to generate a Python program that:

- Implements Binary Search using recursion
- Searches for a given element in a sorted list
- Includes:
 - o Clear inline comments
 - o A step-by-step explanation of the recursive logic

After generating the code, analyze:

- Whether the explanation clearly describes the base case and recursive case case
- Whether the comments correctly match the code logic
- Whether the code is understandable for beginner-level students

Prompt Used: Generate a Python program for recursive binary search. Include clear comments explaining the base case and recursive case.

```
def recursive_binary_search(arr, low, high, x):
    # Base Case: If the search space is empty, the element is not present
    if high < low:
        return -1

    mid = (low + high) // 2

    # Base Case: If element is present at the middle itself
    if arr[mid] == x:
        return mid

    # Recursive Case: If element is smaller than mid, then it can only be present in left subarray
    elif arr[mid] > x:
        return recursive_binary_search(arr, low, mid - 1, x)

    # Recursive Case: Else the element can only be present in right subarray
    else:
        return recursive_binary_search(arr, mid + 1, high, x)

# Example usage:
if __name__ == "__main__":
    my_list = [2, 3, 4, 10, 40]
    target = 10

    # Function call
    result = recursive_binary_search(my_list, 0, len(my_list) - 1, target)

    if result != -1:
        print(f"Element {target} is present at index {result}")
    else:
        print(f"Element {target} is not present in array")

    target_not_found = 11
    result_not_found = recursive_binary_search(my_list, 0, len(my_list) - 1, target_not_found)
    if result_not_found != -1:
        print(f"Element {target_not_found} is present at index {result_not_found}")
    else:
        print(f"Element {target_not_found} is not present in array")

Element 10 is present at index 3
Element 11 is not present in array
```

Gemini

Generate a Python program for recursive binary search. Include clear comments explaining the base case and recursive case.

Certainly! Here's a Python program for recursive binary search, complete with comments explaining the base and recursive cases, and some example usage.

The recursive binary search program executed successfully! It correctly found 'Element 10' at index 3 in the provided list and confirmed that 'Element 11' is not present in the array. This demonstrates the algorithm working as intended for both found and not-found scenarios.

Explain the time complexity of the recursive binary search Add more test cases to the loan approval system Refactor the loan approval system to use a dictionary for criteria

What can I help you build?

+ Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Explanation: In this task, the recursive binary search code is easy to understand because clear comments explain what each part does. The base case and recursive calls are clearly mentioned, so it is simple to follow how the algorithm works step by step. This makes the program transparent and helps users trust the logic and results.

Task - 4:

Ethical Evaluation of AI-Based Scoring Systems

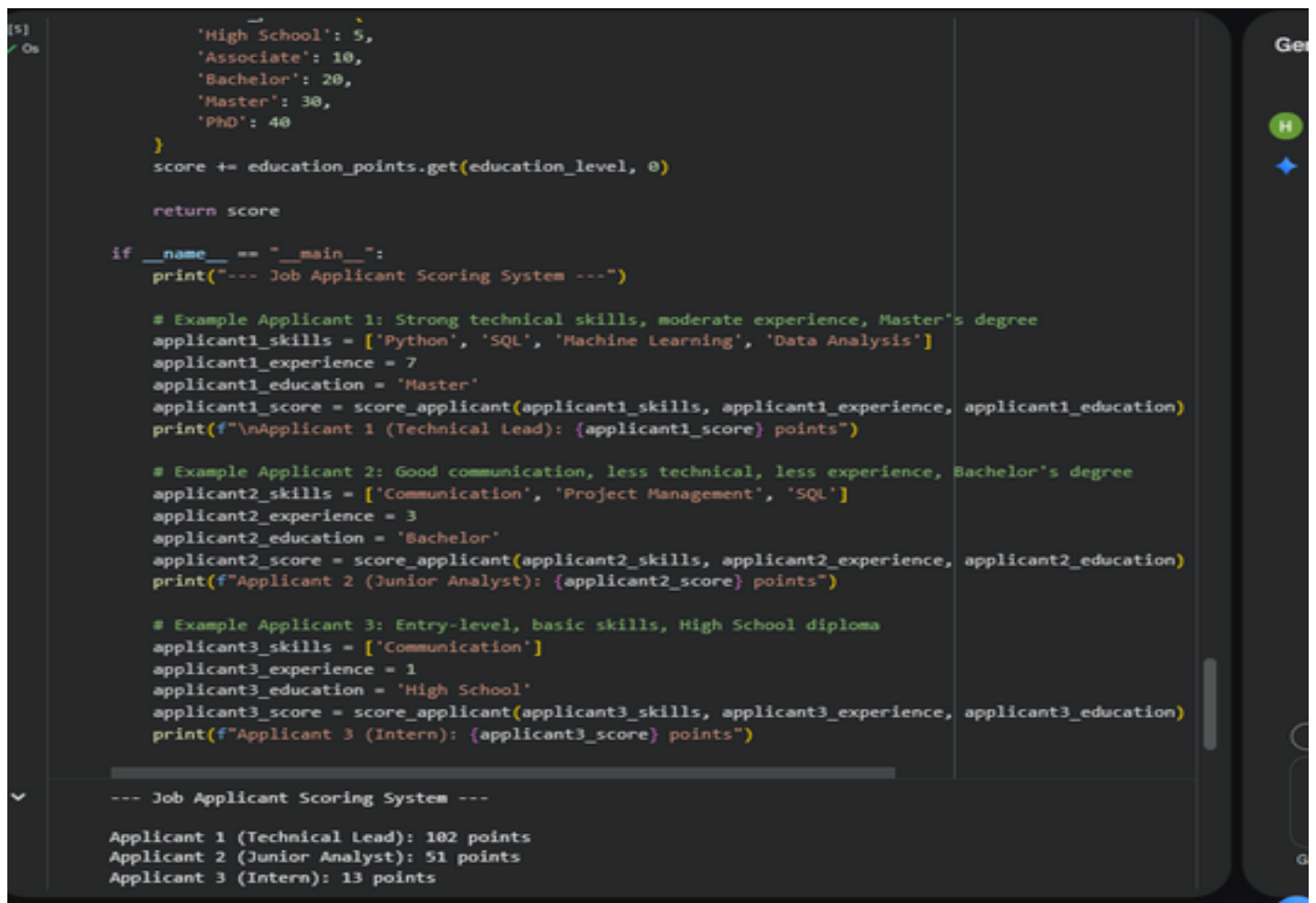
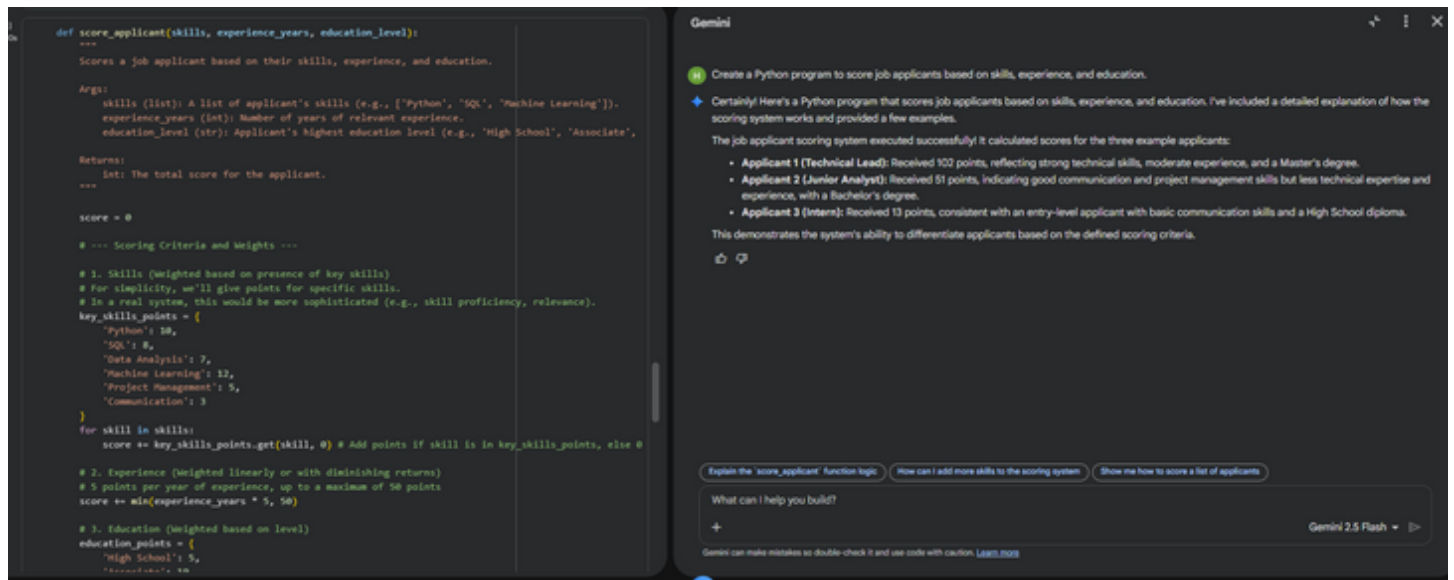
Ask an AI tool to generate a job applicant scoring system based on features such as:

- Skills
- Experience
- Education

Analyze the generated code to check:

- Whether gender, name, or unrelated features influence scoring
- Whether the logic is fair and objective

Prompt Used: Create a Python program to score job applicants based on skills, experience, and education.



Explanation: In this task, the job applicant scoring system evaluates candidates based only on skills, experience, and education, which are directly related to job performance. The program does not use personal details like name, gender, or age, so it avoids unfair bias. Since the scoring criteria are clearly defined and job-relevant, the system follows ethical principles and promotes fair decision-making.

Task - 5:

Inclusiveness and Ethical Variable Design

Use an AI tool to generate a Python code snippet that processes user or employee details.

Analyze the code to identify:

- Gender-specific variables (e.g., male, female)
- Assumptions based on gender or identity
- Non-inclusive naming or logic

Modify or regenerate the code to:

- Use gender-neutral variable names
- Avoid gender-based conditions unless strictly required
- Ensure inclusive and respectful coding practices

Prompt Used: Create a Python program that processes employee details such as name, age, gender, and salary.

The image shows a code editor on the left and a Gemini AI chat interface on the right. The code editor contains a Python program for an Employee Management System. The program defines an `Employee` class with attributes `name`, `age`, `gender`, and `salary`. It includes methods for adding employees to a list and displaying their details. The `__main__` block initializes an empty list for employees, adds three example employees (Alice Smith, Bob Johnson, and Charlie Brown), and displays their details. The Gemini chat interface on the right shows the prompt "Create a Python program that processes employee details such as name, age, gender, and salary." and the AI's response, which confirms the program's execution and displays the details of the three example employees.

```
[14] ✓ 0s  
class Employee:  
    def __init__(self, name, age, gender, salary):  
        self.name = name  
        self.age = age  
        self.gender = gender  
        self.salary = salary  
  
    def display_details(self):  
        print(f"Name: {self.name}, Age: {self.age}, Gender: {self.gender}, Salary: ${self.salary:,.2f}")  
  
def add_employee(employees_list, name, age, gender, salary):  
    employee = Employee(name, age, gender, salary)  
    employees_list.append(employee)  
    print(f"Employee {name} added successfully!")  
  
def display_all_employees(employees_list):  
    if not employees_list:  
        print("No employees to display.")  
        return  
    print("Employee list ---")  
    for emp in employees_list:  
        emp.display_details()  
    print("-----")  
  
if __name__ == "__main__":  
    employees = [] # This list will store Employee objects  
    print("Employee Management System ---")  
  
    # Adding some example employees  
    add_employee(employees, "Alice Smith", 30, "Female", 60000)  
    add_employee(employees, "Bob Johnson", 45, "Male", 75000)  
    add_employee(employees, "Charlie Brown", 25, "Non-binary", 50000)  
  
    # Displaying all employees  
    display_all_employees(employees)  
  
    # You can also add employees interactively:  
    while True:
```

Gemini

Create a Python program that processes employee details such as name, age, gender, and salary.

Certainly! Here's a Python program that uses a class to process and manage employee details like name, age, gender, and salary. You can add new employees and display their information.

The employee management system executed successfully! It added three example employees (Alice Smith, Bob Johnson, and Charlie Brown) and then displayed their details, including their names, ages, genders, and salaries, just as intended. This confirms the system is working correctly.

What can I help you build?

Gemini 2.5 Flash


```
[6]
✓ Os

# Adding some example employees
add_employee(employees, "Alice Smith", 30, "Female", 60000)
add_employee(employees, "Bob Johnson", 45, "Male", 75000)
add_employee(employees, "Charlie Brown", 25, "Non-binary", 50000)

# Displaying all employees
display_all_employees(employees)

# You can also add employees interactively:
# while True:
#     name = input("Enter employee name (or 'q' to quit): ")
#     if name.lower() == 'q':
#         break
#     age = int(input("Enter age: "))
#     gender = input("Enter gender: ")
#     salary = float(input("Enter salary: "))
#     add_employee(employees, name, age, gender, salary)
#     display_all_employees(employees)

--- Employee Management System ---
Employee Alice Smith added successfully!
Employee Bob Johnson added successfully!
Employee Charlie Brown added successfully!

--- Employee List ---
Name: Alice Smith, Age: 30, Gender: Female, Salary: $60,000.00
Name: Bob Johnson, Age: 45, Gender: Male, Salary: $75,000.00
Name: Charlie Brown, Age: 25, Gender: Non-binary, Salary: $50,000.00
-----
```

Explanation: In this task, the employee management system processes details like name, age, gender, and salary without making decisions based on gender. Although gender information is collected, it is not used to affect salary or employee handling, which helps avoid bias. The program treats all employees equally and uses neutral logic, making it more inclusive and ethically designed.